

# Spring Boot Cloud Configuration Server

Informe del taller: implementación y documentación

**Camilo Andrés Vargas Ríos**

Repositorio en GitHub
`https://github.com/camilovargas-05/taller-spring-cloud-config`

9 de octubre de 2026

# Contenido

1. Introducción y objetivo

2. Marco conceptual

3. Herramientas y entorno

4. Arquitectura de la solución

5. Parte 1: servidor de configuración (config-server)

   5.1 Creación del proyecto · 5.2 Dependencias · 5.3 Clase principal · 5.4 Configuración · 5.5 Archivos por perfil · 5.6 Ejecución · 5.7 Verificación

6. Parte 2: cliente de configuración (loan-service)

   6.1 Creación del proyecto · 6.2 Dependencias · 6.3 Configuración · 6.4 Controlador REST · 6.5 Ejecución y prueba

7. Pruebas con distintos perfiles

8. Problemas encontrados y soluciones

9. Repositorio en GitHub

10. Conclusiones

11. Referencias

# 1. Introducción y objetivo

En una arquitectura de microservicios cada servicio necesita su propia configuración: puertos, URLs, credenciales, mensajes, etc. Si esa configuración vive dentro de cada aplicación, cambiarla obliga a modificar el código y volver a desplegar el servicio. **Spring Cloud Config** resuelve este problema centralizando la configuración en un servidor independiente.

**Objetivo del taller:** implementar, siguiendo paso a paso el tutorial "*Spring Boot – Cloud Configuration Server*" de GeeksforGeeks, un **servidor de configuración** y un **microservicio cliente** ( `loan-service` ). El cliente obtiene su configuración del servidor y cambia de comportamiento según el perfil activo ( `default` , `dev` o `uat` ).

# 2. Marco conceptual

| Concepto | Descripción |
|---|---|
| **Configuración externalizada** | Práctica de sacar la configuración del código para poder cambiarla sin recompilar la aplicación. |
| **Config Server** | Aplicación Spring Boot anotada con `@EnableConfigServer`. Expone una API REST ( `/{aplicación}/{perfil}` ) que devuelve las propiedades de cada aplicación y perfil. |
| **Config Client** | Aplicación que, al arrancar, consulta el Config Server (mediante `spring.config.import` ) y carga esas propiedades en su `Environment`. |
| **Perfiles (profiles)** | Permiten tener valores distintos por entorno. El archivo `loan-service-dev.properties` solo se aplica cuando el perfil activo es `dev`. El archivo base `loan-service.properties` se aplica siempre y tiene menor prioridad. |
| **Backend *native*** | Modo en el que el Config Server lee los archivos del sistema de archivos o del *classpath*, en lugar de un repositorio Git. Es el modo que usa el tutorial. |
| **Spring Boot DevTools** | Herramienta de desarrollo que reinicia la aplicación automáticamente cuando cambian los archivos. |

# 3. Herramientas y entorno

| Herramienta | Versión / detalle |
|---|---|
| Sistema operativo | Windows 11 |
| Java (JDK) | 21.0.9 LTS |
| Spring Boot | 4.1.1 |
| Spring Cloud | 2025.1.3 |
| Gestor de dependencias | Maven, mediante el *Maven Wrapper* ( `mvnw` ) incluido en cada proyecto |
| Generador de proyectos | Spring Initializr (start.spring.io) |
| Control de versiones | Git 2.45 y GitHub |
| Pruebas | Navegador web (Google Chrome) y `curl` |

> **Nota sobre versiones:** el tutorial pide Java 17 o superior. Se usó Java 21 y la versión estable más reciente de Spring Boot que ofrecía Spring Initializr al momento del taller (4.1.1). Los pasos y las anotaciones son los mismos del tutorial. El tutorial menciona MySQL como requisito, pero el ejemplo no lo usa, así que no se instaló.

# 4. Arquitectura de la solución




El **config-server** arranca en el puerto 8888 y publica los archivos de la carpeta `config/`. Al iniciar, **loan-service** se identifica con su nombre (`spring.application.name=loan-service`) y su perfil activo, y pide al servidor sus propiedades. El servidor le devuelve, entre otras, `server.port` y `application.message`. Por eso el puerto donde arranca el cliente lo decide el servidor y no el propio cliente.

# 5. Parte 1: servidor de configuración (config-server)

## 5.1 Creación del proyecto con Spring Initializr

En `start.spring.io` se creó un proyecto Maven con Java, empaquetado JAR y Java 21, usando los metadatos `com.taller` / `config-server`. Se agregaron las tres dependencias que indica el tutorial: **Spring Web**, **Spring Boot DevTools** y **Config Server**.




**Figura 1.** Configuración del proyecto `config-server` en Spring Initializr.

## 5.2 Dependencias (pom.xml)

Initializr agrega el BOM de Spring Cloud (`spring-cloud-dependencies`, versión 2025.1.3) y la dependencia `spring-cloud-config-server`:

config-server/pom.xml (fragmento)

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-config-server</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

## 5.3 Clase principal con @EnableConfigServer

La anotación `@EnableConfigServer` convierte la aplicación Spring Boot en un servidor de configuración. Es el único cambio necesario en el código Java del servidor.

config-server/src/main/java/com/taller/configserver/ConfigServerApplication.java

```
package com.taller.configserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.config.server.EnableConfigServer;

@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }

}
```

## 5.4 Configuración del servidor (application.properties)

**config-server/src/main/resources/application.properties**

```
spring.application.name=config-server
server.port=8888

# Perfil "native": las configuraciones se leen del sistema de archivos / classpath
spring.profiles.active=native
spring.cloud.config.server.native.search-locations=classpath:/config
```

- `server.port=8888` : puerto del servidor de configuración. Ver la sección 8.
- `spring.profiles.active=native` : activa el backend *native*, que lee archivos locales en lugar de un repositorio Git.
- `spring.cloud.config.server.native.search-locations=classpath:/config` : carpeta donde el servidor busca los archivos de configuración.

## 5.5 Archivos de configuración por perfil

Dentro de `src/main/resources/config/` se crearon tres archivos. El nombre sigue la convención `{spring.application.name}-{perfil}.properties` :

**config-server/src/main/resources/config/loan-service.properties**

```
server.port=8081
application.message=Hola desde Loan Service - perfil DEFAULT (servido por Config Server)
```

**config-server/src/main/resources/config/loan-service-dev.properties**

```
server.port=8082
application.message=Hola desde Loan Service - perfil DEV (servido por Config Server)
```

**config-server/src/main/resources/config/loan-service-uat.properties**

```
server.port=8083
application.message=Hola desde Loan Service - perfil UAT (servido por Config Server)
```

## 5.6 Ejecución del servidor

Desde la carpeta `config-server` se ejecutó `./mvnw spring-boot:run`, que equivale al `mvn spring-boot:run` del tutorial pero usa el Maven Wrapper. En la consola se ve que el perfil `native` está activo y que Tomcat arrancó en el puerto **8888**:

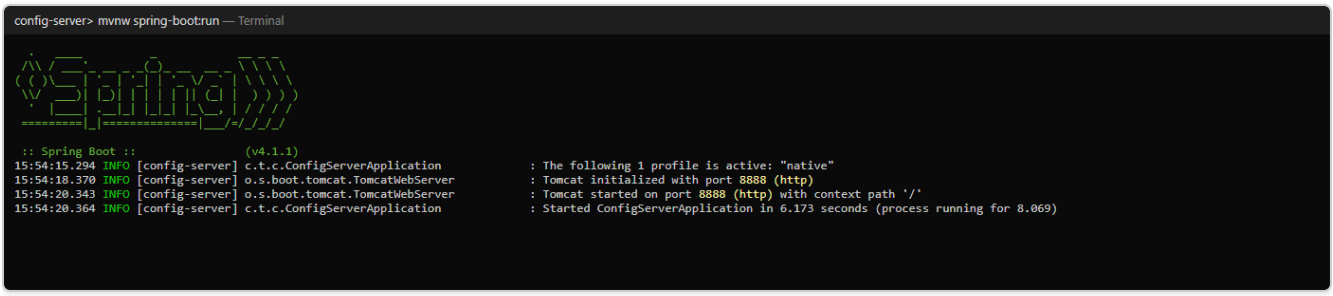



**Figura 2.** Arranque del Config Server: perfil `native` y puerto 8888.

## 5.7 Verificación de los endpoints del servidor

El Config Server expone cada configuración en `http://localhost:8888/{aplicación}/{perfil}`. Se consultaron los tres perfiles desde el navegador:

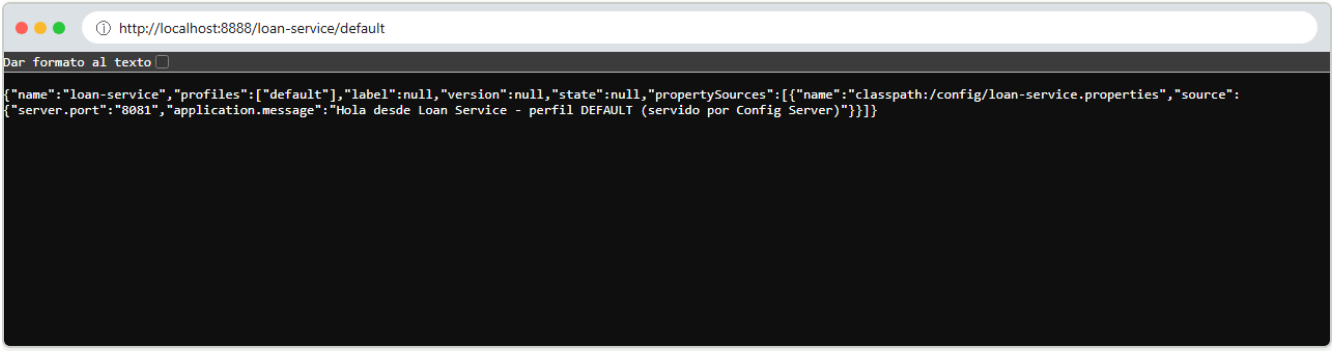



**Figura 3.** `/loan-service/default`: solo devuelve el archivo base (puerto 8081).

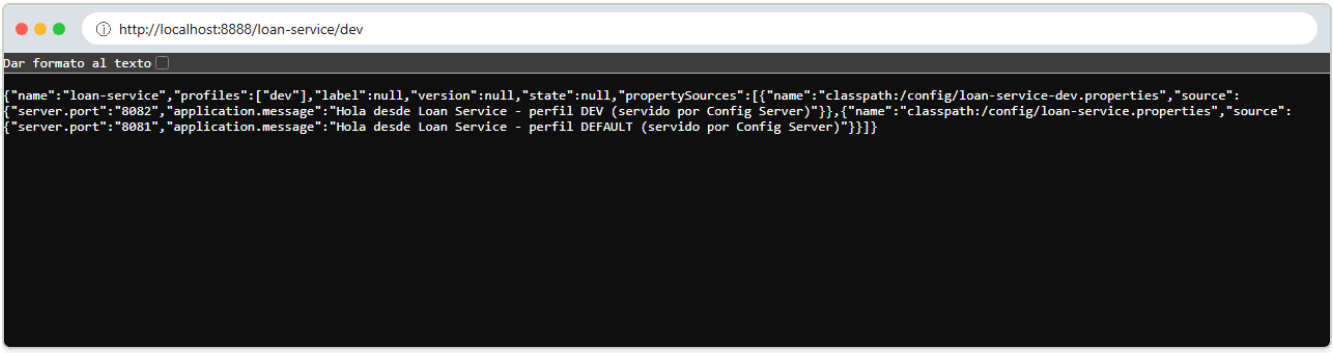



**Figura 4.** `/loan-service/dev`: devuelve `loan-service-dev.properties` (8082) y, con menor prioridad, el archivo base.

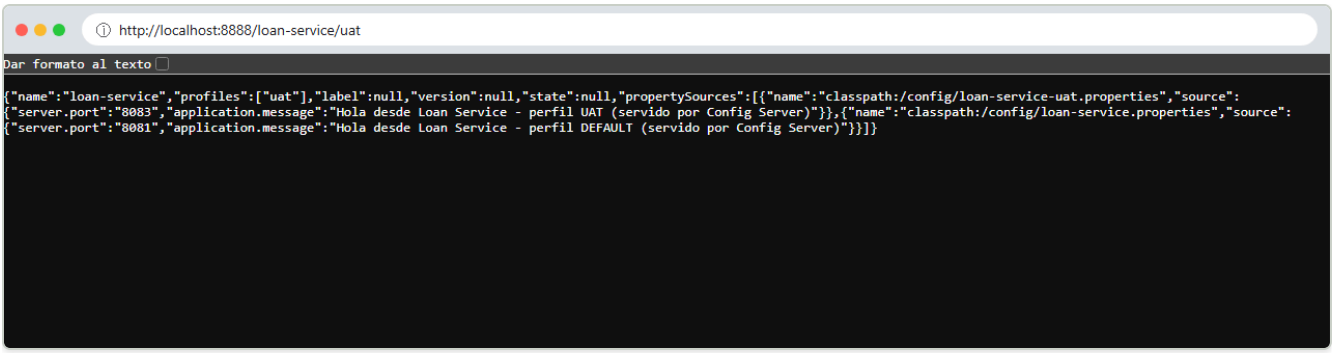



**Figura 5.** `/loan-service/uat`: devuelve `loan-service-uat.properties` (8083) y el archivo base.

> El servidor responde correctamente. En `propertySources` se ve el orden de prioridad: primero el archivo del perfil y después el archivo base.

# 6. Parte 2: cliente de configuración (loan-service)

## 6.1 Creación del proyecto

Se generó un segundo proyecto, `loan-service`, con **Spring Web**, **Spring Boot DevTools** y **Config Client**:




**Figura 6.** Configuración del proyecto `loan-service` en Spring Initializr.

## 6.2 Dependencias (pom.xml)

**loan-service/pom.xml (fragmento)**

```xml
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-config</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

## 6.3 Configuración del cliente (application.properties)

**loan-service/src/main/resources/application.properties**

```properties
spring.application.name=loan-service

# Importa la configuracion desde el Config Server
spring.config.import=optional:configserver:http://localhost:8888

# Perfil activo (default | dev | uat)
spring.profiles.active=dev
```

- `spring.application.name=loan-service`: con este nombre el servidor sabe qué archivos `loan-service*.properties` devolver.
- `spring.config.import=optional:configserver:http://localhost:8888`: indica al cliente que importe su configuración del Config Server. Con el prefijo `optional:` la aplicación intenta arrancar aunque el servidor no esté disponible.
- `spring.profiles.active=dev`: perfil activo. Decide qué archivo de perfil se aplica.

El cliente **no** define `server.port` ni `application.message`. Ambos valores llegan del servidor.

## 6.4 Clase principal y controlador REST

**loan-service/src/main/java/com/taller/loanservice/LoanServiceApplication.java**

```java
package com.taller.loanservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class LoanServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(LoanServiceApplication.class, args);
    }

}
```

El controlador inyecta la propiedad remota `application.message` con `@Value` y la expone en `GET /message`:

**loan-service/src/main/java/com/taller/loanservice/LoanController.java**

```java
package com.taller.loanservice;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class LoanController {

    @Value("${application.message}")
    private String message;

    @GetMapping("/message")
    public String getMessage() {
        return message;
    }

}
```

## 6.5 Ejecución y prueba (perfil dev)

Con el servidor en marcha, se ejecutó `./mvnw spring-boot:run` en la carpeta `loan-service`. En la consola se ve que el cliente se conecta a `http://localhost:8888`, carga los entornos `default` y `dev` y arranca en el puerto **8082**, el valor que definió el servidor:

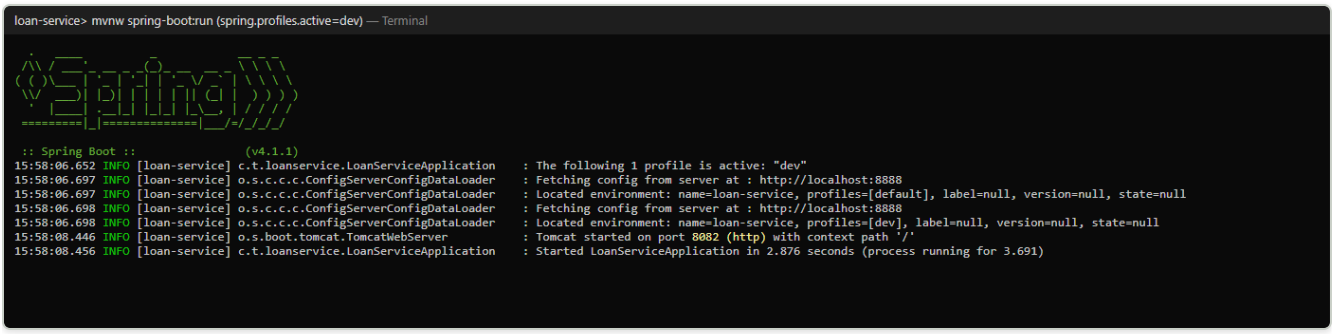



**Figura 7.** Arranque de `loan-service` con el perfil `dev`: obtiene la configuración del servidor y arranca en el puerto 8082.




**Figura 8.** Respuesta de `GET http://localhost:8082/message` con el perfil `dev`.

> El mensaje mostrado viene de `loan-service-dev.properties`, que está en el servidor y no en el cliente. La configuración centralizada funciona.

# 7. Pruebas con distintos perfiles

Como pide el tutorial, se cambió el valor de `spring.profiles.active` en el cliente y se reinició la aplicación, sin tocar el código Java.

### Perfil uat

**loan-service/src/main/resources/application.properties (cambio)**

```
spring.profiles.active=uat
```

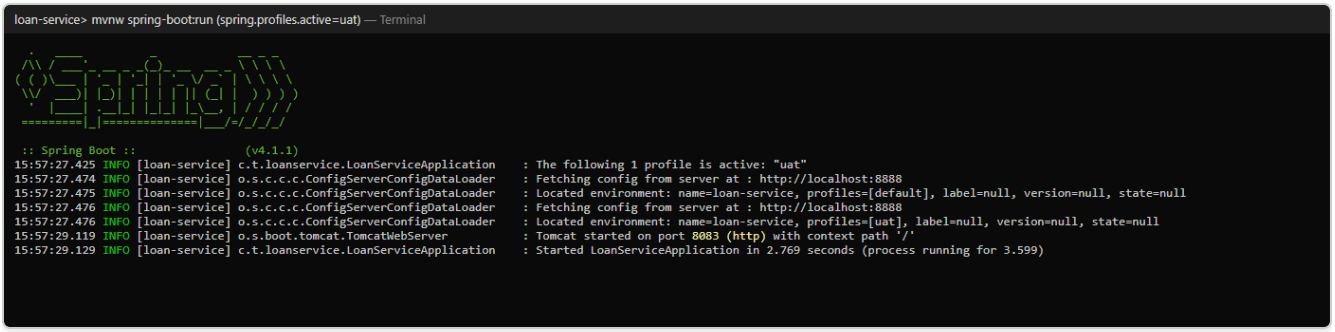



**Figura 9.** Arranque con el perfil `uat`: carga el entorno `uat` y arranca en el puerto 8083.




**Figura 10.** Respuesta de `GET http://localhost:8083/message` con el perfil `uat`.

### Sin perfil (default)

**loan-service/src/main/resources/application.properties (cambio)**

```
#spring.profiles.active=dev
```

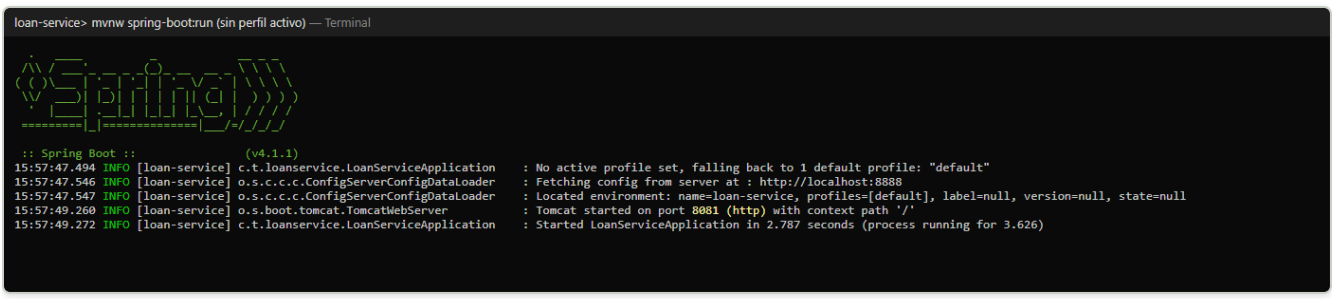



**Figura 11.** Sin perfil activo, Spring usa `default`: arranca en el puerto 8081.

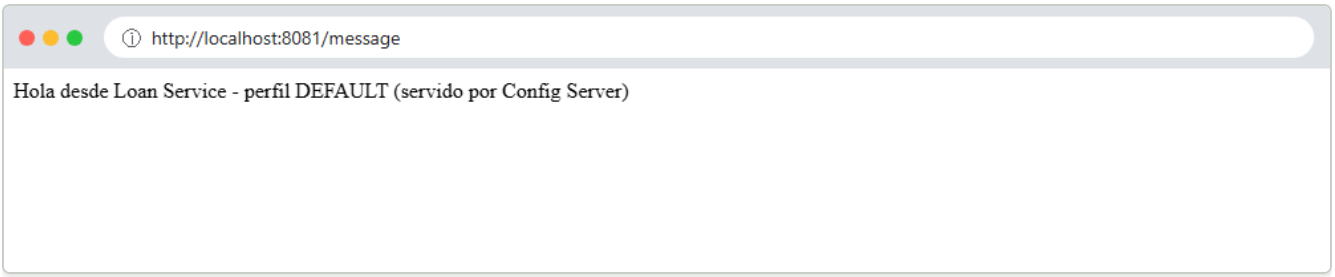



**Figura 12.** Respuesta de `GET http://localhost:8081/message` con el perfil `default`.

### Resumen de resultados

| Perfil activo | Archivo aplicado en el servidor | Puerto | Respuesta de /message | Resultado |
|---|---|---|---|---|
| *ninguno* (default) | loan-service.properties | 8081 | Hola desde Loan Service - perfil DEFAULT… | ✓ |
| dev | loan-service-dev.properties | 8082 | Hola desde Loan Service - perfil DEV… | ✓ |
| uat | loan-service-uat.properties | 8083 | Hola desde Loan Service - perfil UAT… | ✓ |

Al terminar las pruebas, el perfil del cliente se volvió a dejar en `dev`, como en el tutorial.

# 8. Problemas encontrados y soluciones

### 8.1 Puertos inconsistentes en el tutorial

El tutorial configura el servidor con `server.port=8080`, pero el cliente apunta a `http://localhost:8888`. Si se copia tal cual, el cliente no encuentra el servidor y, como la importación es `optional:`, arranca sin la configuración remota. Entonces falla con `Could not resolve placeholder 'application.message'`.

**Solución:** el servidor se configuró en el puerto **8888**, el puerto por defecto de Spring Cloud Config, para que coincida con el cliente.

### 8.2 Prueba unitaria del cliente

Initializr genera una prueba `contextLoads()` que levanta el contexto de Spring. Durante `mvnw package` el Config Server no está disponible, así que la propiedad `application.message` no existe y la prueba falla.

**Solución:** la prueba define un valor de ejemplo para esa propiedad:

**loan-service/src/test/java/com/taller/loanservice/LoanServiceApplicationTests.java**

```
package com.taller.loanservice;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest(properties = "application.message=test")
class LoanServiceApplicationTests {

    @Test
    void contextLoads() {
    }

}
```

### 8.3 Reinicio automático de DevTools al cambiar de perfil

Con la aplicación en ejecución, al editar `spring.profiles.active` DevTools reinició el cliente automáticamente con el nuevo perfil. Si después se lanza otra instancia, esta falla con "*Port 8083 was already in use*". **Solución:** detener el cliente antes de cambiar de perfil y volver a arrancarlo, o aprovechar el reinicio automático de DevTools.

# 9. Repositorio en GitHub

El código fuente de los dos proyectos, las capturas, los logs de ejecución y este informe están publicados en:

**https://github.com/camilovargas-05/taller-spring-cloud-config**

Estructura del repositorio:

```
Estructura
```

```
Taller_Spring_Cloud_Config/
├── README.md                  instrucciones de ejecución
├── config-server/             proyecto del servidor de configuración
├── loan-service/              proyecto del cliente (microservicio)
└── docs/
    ├── Informe_Taller_Spring_Cloud_Config.pdf
    ├── capturas/              imágenes usadas en este informe
    └── logs/                  salida de consola de cada ejecución
```

# 10. Conclusiones

- Spring Cloud Config permite **separar la configuración del código**: el microservicio `loan-service` no contiene ni su puerto ni su mensaje, porque los recibe del servidor al arrancar.
- Con los **perfiles** se puede usar el mismo artefacto en varios entornos (dev, uat, producción) cambiando un solo parámetro, sin recompilar.
- Para levantar un servidor de configuración basta con una anotación ( `@EnableConfigServer` ) y unas pocas propiedades. El cliente solo necesita `spring.config.import`.
- El backend *native* sirve para aprender y desarrollar. En producción se recomienda usar un repositorio **Git** como backend, para versionar la configuración y poder cambiarla sin volver a desplegar el servidor.
- Al seguir un tutorial hay que validarlo: la diferencia de puertos (8080 frente a 8888) habría impedido que el ejemplo funcionara.

# 11. Referencias


- GeeksforGeeks. *Spring Boot – Cloud Configuration Server*. https://www.geeksforgeeks.org/advance-java/spring-boot-cloud-configuration-server/
- Spring. *Spring Cloud Config – Reference Documentation*. https://docs.spring.io/spring-cloud-config/reference/
- Spring Initializr. https://start.spring.io